Project on Fuff

Submitted By:

Name : Joyanta Kumer Sarker

Submitted To:

Name : Navid Bin Mahamud

Associate manager

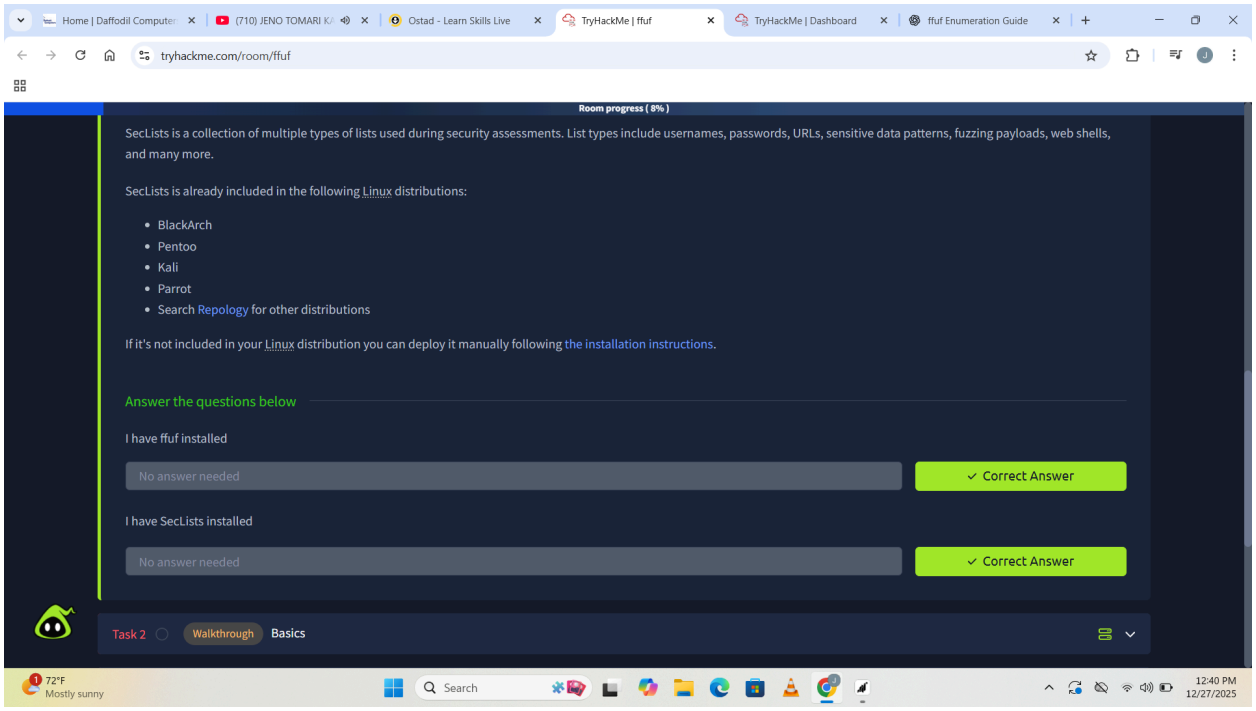
(software security & Risks)

Brac Bank PLC

Submission Date: 28.12.2025

Table of Contents

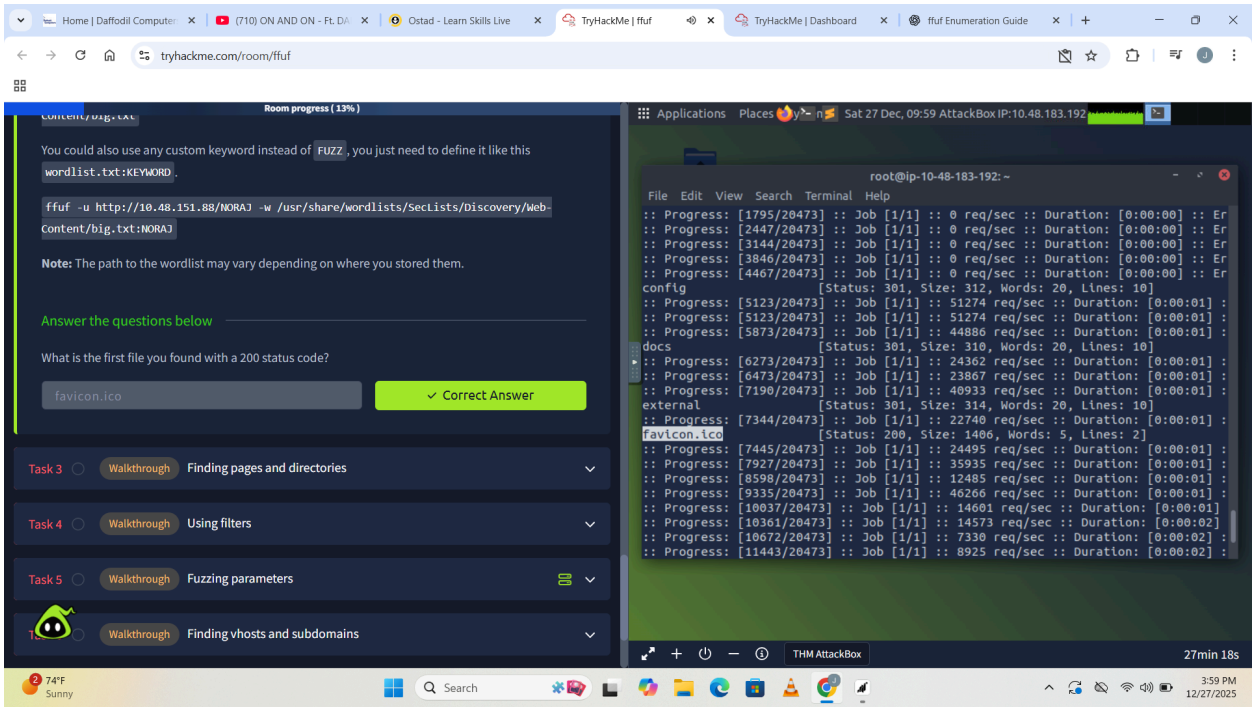
Task 1:



This task introduces ffuf, secLists a web fuzzing tool used to discover hidden files, directories, and parameters on a web server. No command is required; Just need to understand what ffuf does and why it is useful in web enumeration.

Task 2:

Command: ffuf -u http://10.48.151.88/FUZZ -w /usr/share/wordlists/SecLists/Discovery/Web-Content/big.txt



The file **favicon.ico** is the first file found with a **200 status code**, which means the web server successfully served the file and confirmed that it exists and is accessible.

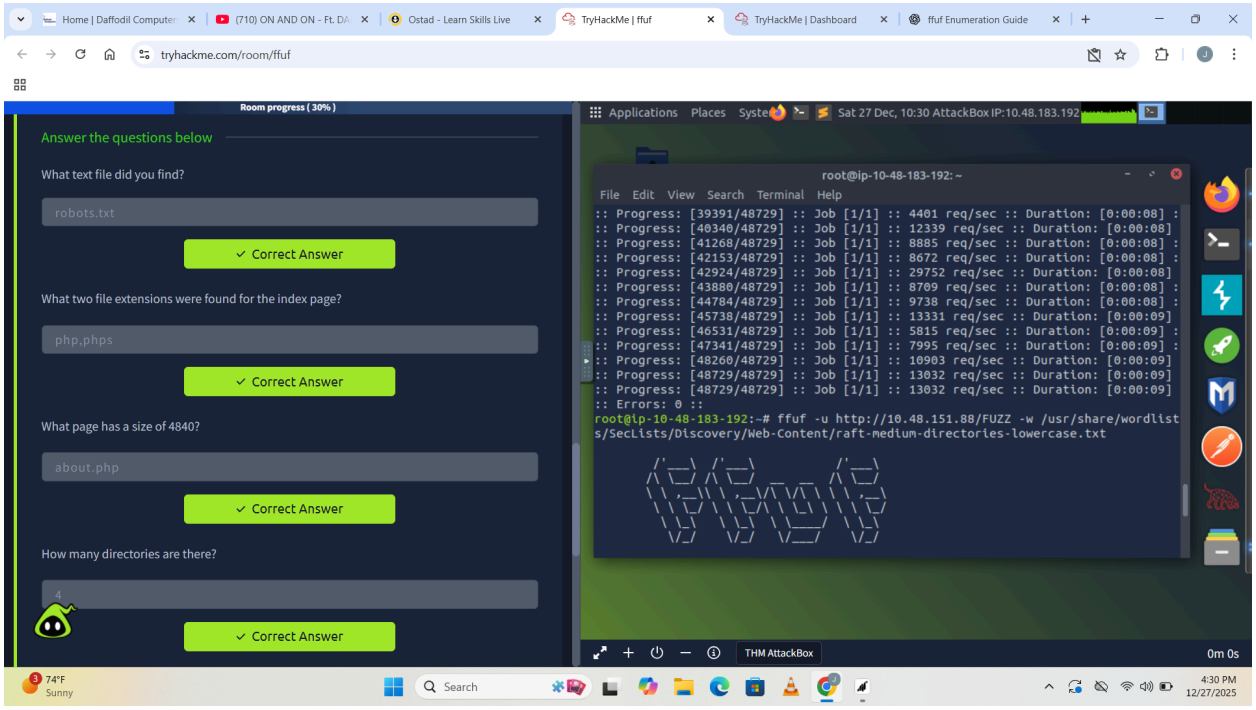
Task 3:

Command 1:ffuf -u http://10.48.151.88/FUZZ -w /usr/share/wordlists/SecLists/Discovery/Web-Content/raft-medium-files-lowercase.txt

Command 2:ffuf -u http://10.48,151.88/indexFUZZ -w /usr/share/wordlists/SecLists/Discovery/Web-Content/web-extensions.txt

Command 3:ffuf -u http://10.48.151.88/FUZZ -w /usr/share/wordlists/secLists/Discovery/Web-Content/raft-medium-words-lowercase.txt -e .php,.txt

Command 4:ffuf -u http://10.48.151.88/FUZZ -w /usr/share/wordlists/SecLists/Discovery/Web-Content/raft-medium-directories-lowercase.txt



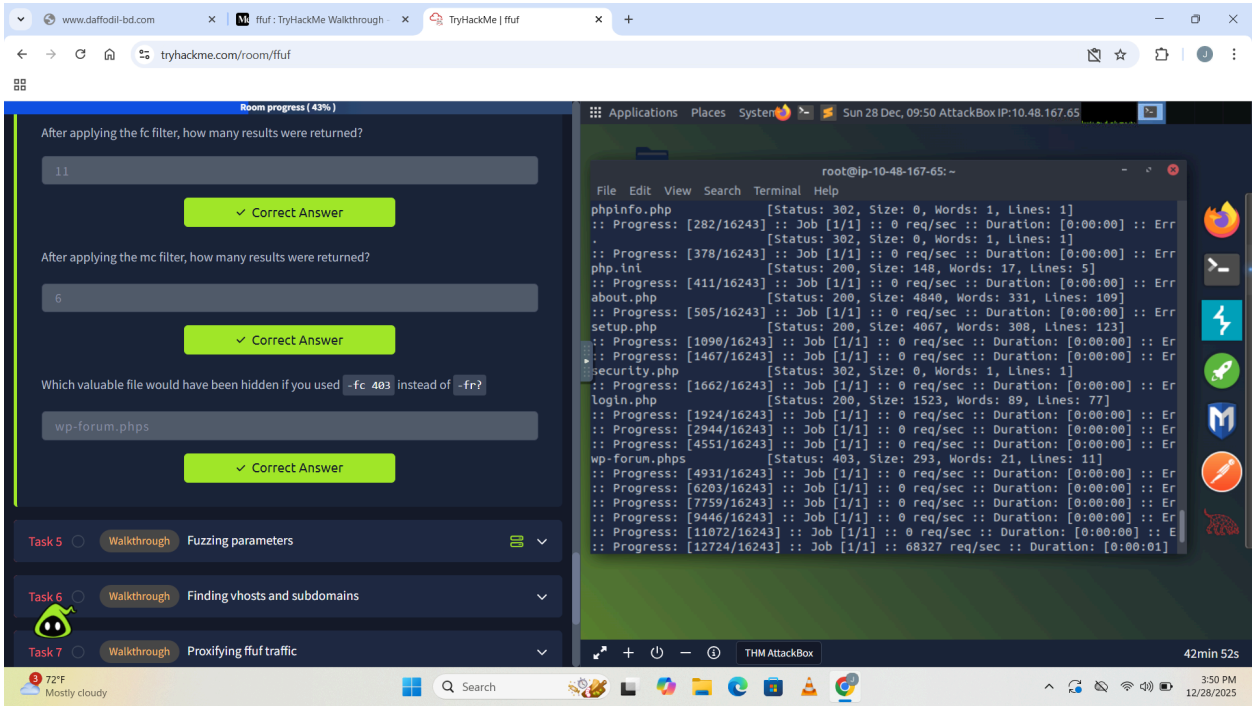
During web enumeration with ffuf, the text file **robots.txt** was discovered, which often reveals restricted or hidden paths. The index page was found to have two file extensions, **php** and **phps**, indicating different ways the page is served. The page with a response size of **4840 bytes** was identified as **about.php**, showing it contains unique content. Finally, directory brute forcing revealed a total of **4 directories**, indicating multiple accessible paths on the web server.

Task -4

command 1: ffuf -u http://10.48.175.119/FUZZ -w /usr/share/wordlists/SecLists/Discovery/Web-Content/raft-medium-files-lowercase.txt -fc 403

command 2: ffuf -u http://10.48.175.119/FUZZ -w /usr/share/wordlists/SecLists/Discovery/Web-Content/raft-medium-files-lowercase.txt -mc 200

command 3: ffuf -u http://10.48.175.119/FUZZ -w /usr/share/wordlists/SecLists/Discovery/Web-Content/raft-medium-files-lowercase.txt -fr '/\..*'



After applying the **-fc (filter code)** option, ffuf removed all responses with status code **403**, leaving **11 valid results**. When the **-mc (match code)** option was used, ffuf showed only responses with status code **200**, resulting in **6 valid results**.

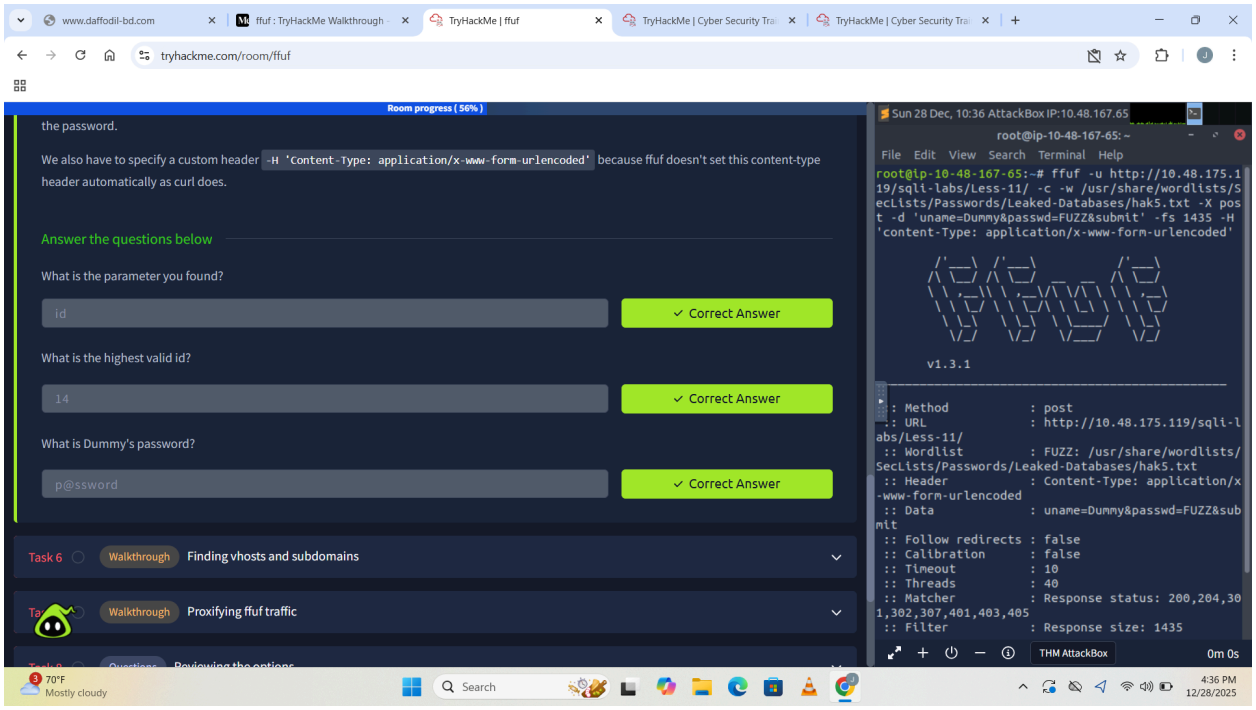
Finally, using **`-fc 403`** instead of **`-fr` (filter regex)** would have hidden the valuable file **`wp-forum.phps`**, because it also returns a 403 status code even though the file exists.

Task 5:

command 1: $ ffuf -u 'http://10.48.175.119/sqli-labs/Less-1/?FUZZ=1' -c -w /usr/share/wordlists/SecLists/Discovery/Web-Content/burp-parameter-names.txt -fw 39

command 2: :~/ffuf# for i in {0..255}; do echo $i; done | ffuf -u 'http://10.48.175.119/sqli-labs/Less-1/?id=FUZZ' -c -w - -fw 33

command 3: :~/ffuf# ffuf -u http://10.48.175.119/sqli-labs/Less-11/ -c -w /usr/share/wordlists/SecLists/Passwords/Leaked-Databases/hak5.txt -X POST -d 'uname=Dummy&passwd=FUZZ&submit=Submit' -fs 1435 -H 'Content-Type: application/x-www-form-urlencoded'



In this task, ffuf is used for parameter fuzzing to discover which parameters a web page accepts. First, the `FUZZ` keyword is placed in the URL to test different parameter names, which reveals `id` as a valid parameter. After identifying that the parameter accepts integer values, ffuf is combined with number generation (via piping stdout using `-w -`) to fuzz values from 0 to 255, allowing us to find the highest valid id, which is 14. Finally, ffuf is used to perform a POST-based brute-force attack on a login form by fuzzing the password field, successfully revealing Dummy's password as `p@ssword`, demonstrating how ffuf can be used for parameter discovery, value testing, and credential brute forcing.

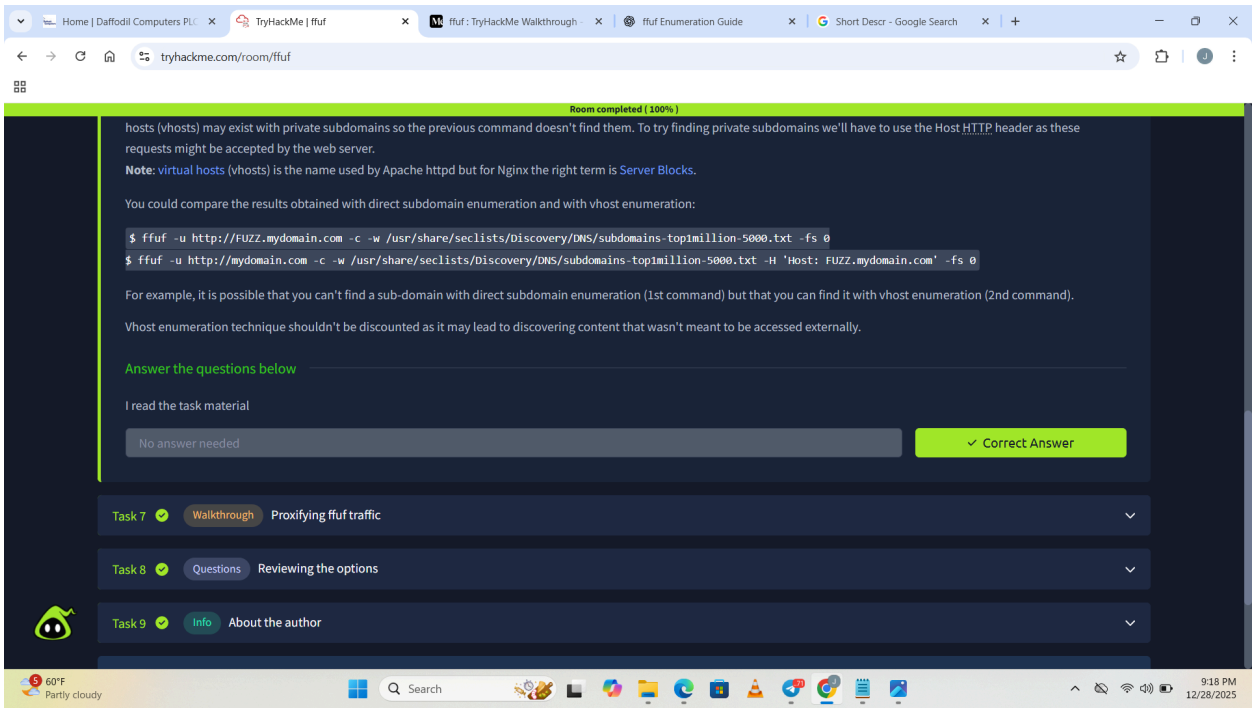
Task 6:

This task explains how **ffuf can be used for subdomain and virtual host (vhost) enumeration**. First, ffuf tries to discover public subdomains by fuzzing the subdomain part of the URL, but this may miss private subdomains that are not resolvable via public DNS. To find those hidden or internal subdomains, ffuf can fuzz the **Host HTTP header**, which the web server may accept even if DNS does

not resolve it. Comparing direct subdomain enumeration with vhost enumeration helps uncover internal or unintended content that should not be publicly accessible.

ffuf -u http://FUZZ.mydomain.com -c -w /usr/share/wordlists/SecLists/Discovery/DNS/subdomains-top1million-5000.txt -fs 0

ffuf -u http://mydomain.com -c -w /usr/share/wordlists/SecLists/Discovery/DNS/subdomains-top1million-5000.txt -H 'Host: FUZZ.mydomain.com' -fs 0



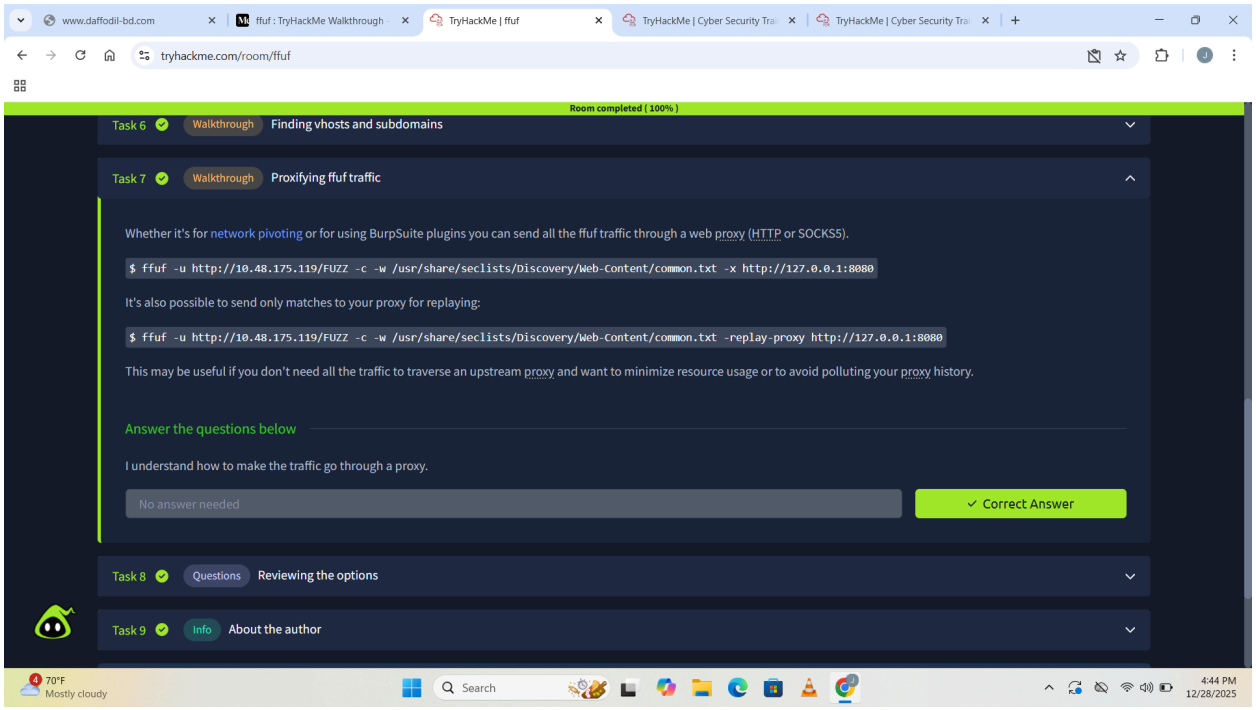
Task 7:

This task explains how to send ffuf traffic through a web proxy such as Burp Suite for inspection, modification, or replaying requests. Using the -x option sends all ffuf requests through the proxy, which is useful for monitoring or using Burp

plugins, while the `-replay-proxy` option sends only matched (interesting) requests to the proxy to reduce noise and save resources. This task is informational only, so no answer is required.

ffuf -u http://MACHINE_IP/FUZZ -c -w /usr/share/wordlists/SecLists/Discovery/Web-Content/common.txt -x http://127.0.0.1:8080

ffuf -u http://MACHINE_IP/FUZZ -c -w /usr/share/wordlists/SecLists/Discovery/Web-Content/common.txt -replay-proxy http://127.0.0.1:8080



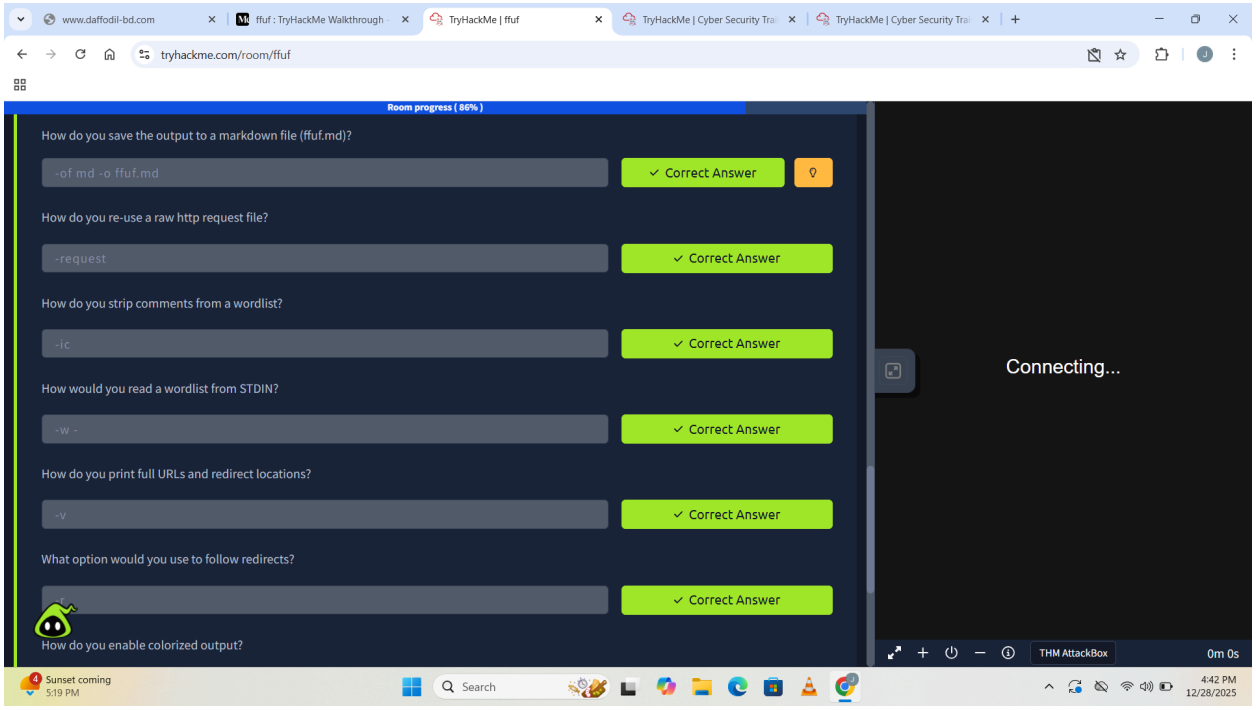
Task 8:

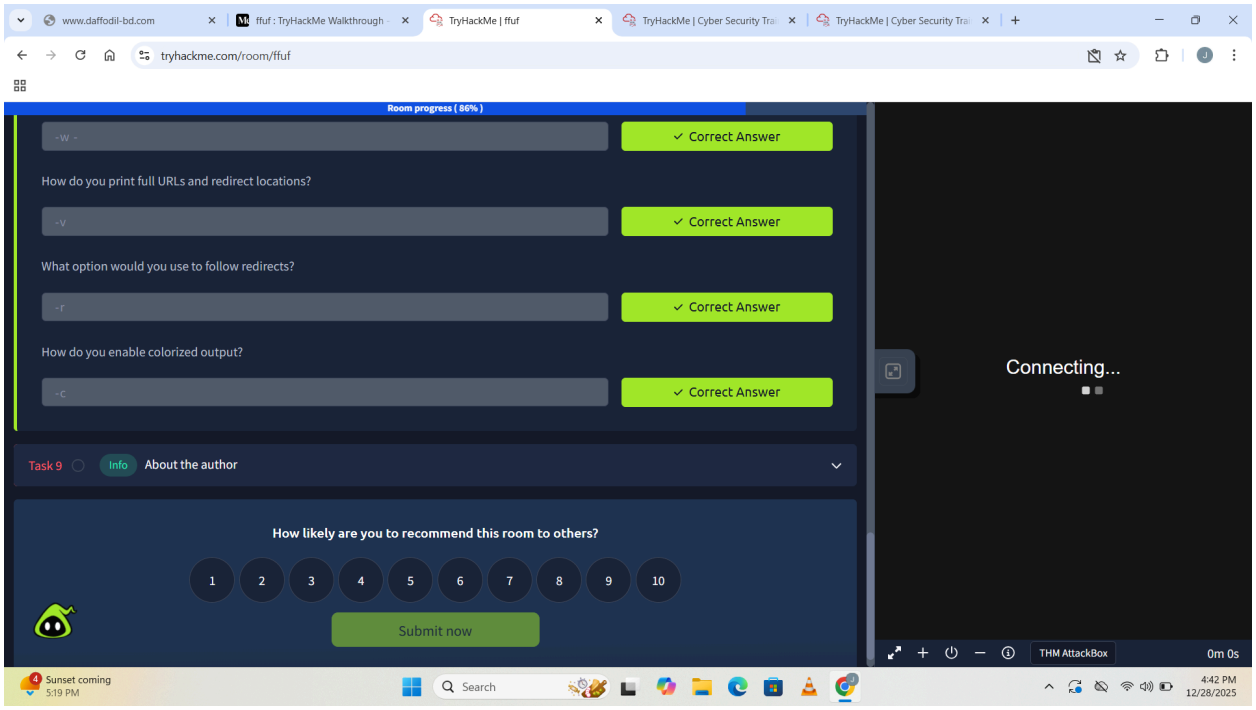
This task focuses on reviewing important ffuf options that make fuzzing more efficient and flexible in real-world scenarios. The -ic option is used to ignore comments in wordlists, which is helpful when wordlists contain headers, copyright

notes, or comments that should not be fuzzed. ffuf also supports saving results in different formats, reusing raw HTTP requests, reading wordlists from standard input, and improving output readability. Using options like -v (verbose), -r (follow redirects), and -c (colorized output) helps analysts better understand responses, track redirects, and quickly identify interesting results. Running ffuf -h allows users to explore many more advanced options depending on the testing situation.

Key Options Used and Answers:

- Save output to a markdown file: -of md -o ffuf.md
- Re-use a raw HTTP request file: -request
- Strip comments from a wordlist: -ic
- Read a wordlist from STDIN: -w -
- Print full URLs and redirect locations: -v
- Follow redirects: -r
- Enable colorized output: -c



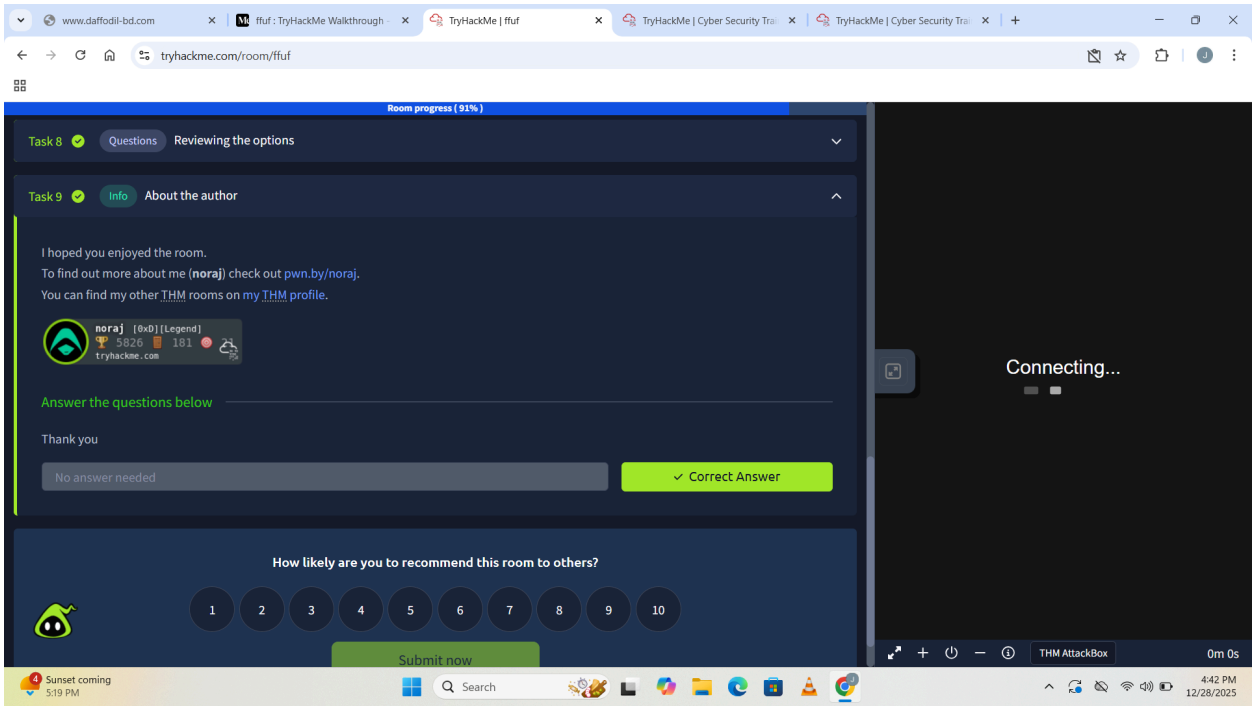


Task 9:

This final section is a closing note from the room author, thanking users for completing the room and inviting them to explore more of the author's work and other TryHackMe rooms. It is informational only and does not require any command or answer.

Command:

No command required.


www.daffodil-bd.com
ffuf : TryHackMe Walkthrough
TryHackMe | ffuf
TryHackMe | Cyber Security Tra
TryHackMe | Cyber Security Tra
tryhackme.com/room/ffuf
Room progress (91%)
Task 8
Questions
Reviewing the options
Task 9
Info
About the author
I hoped you enjoyed the room.
To find out more about me (noraj) check out pwn.by/noraj.
You can find my other THM rooms on my THM profile.
noraj [0xD][Legend]
5826 181
tryhackme.com
Answer the questions below
Thank you
No answer needed
✓ Correct Answer
How likely are you to recommend this room to others?
1 2 3 4 5 6 7 8 9 10
Submit now
Connecting...
THM AttackBox
0m 0s
Sunset coming
5:19 PM
Search
4:42 PM
12/28/2025